True/False Questions (10 pts)
Problem 1.1: False (should be non linear function)
Problem 1.2: False (should computationally heavy but memory efficient)
Problem 1.3: True
Problem 1.4: False
Problem 1.5: True

Adalines (15 pts)

Problem 2.1:

x1	**x2**	**s**	**y**
-1	-1	-3	-1
-1	+1	-1	-1
+1	-1	-1	-1
+1	+1	+1	+1

Logic: AND

Problem 2.2:

x1	**x2**	**s**	**y**
-1	-1	+3	+1
-1	+1	+1	+1
+1	-1	+1	+1
+1	+1	-1	-1

w0 = +1, w1 = -1, w2 = -1

Problem 2.3

x1	**x2**	**x3**	**s**	**y**
-1	-1	-1	-3	-1
-1	-1	+1	-1	-1
-1	+1	-1	-1	-1

-1	+1	+1	+1	+1
+1	-1	-1	-1	-1
+1	-1	+1	+1	+1
+1	+1	-1	+1	+1
+1	+1	+1	+3	+1

w0 = 0, w1 = +1, w2 = +1, w3 = +1

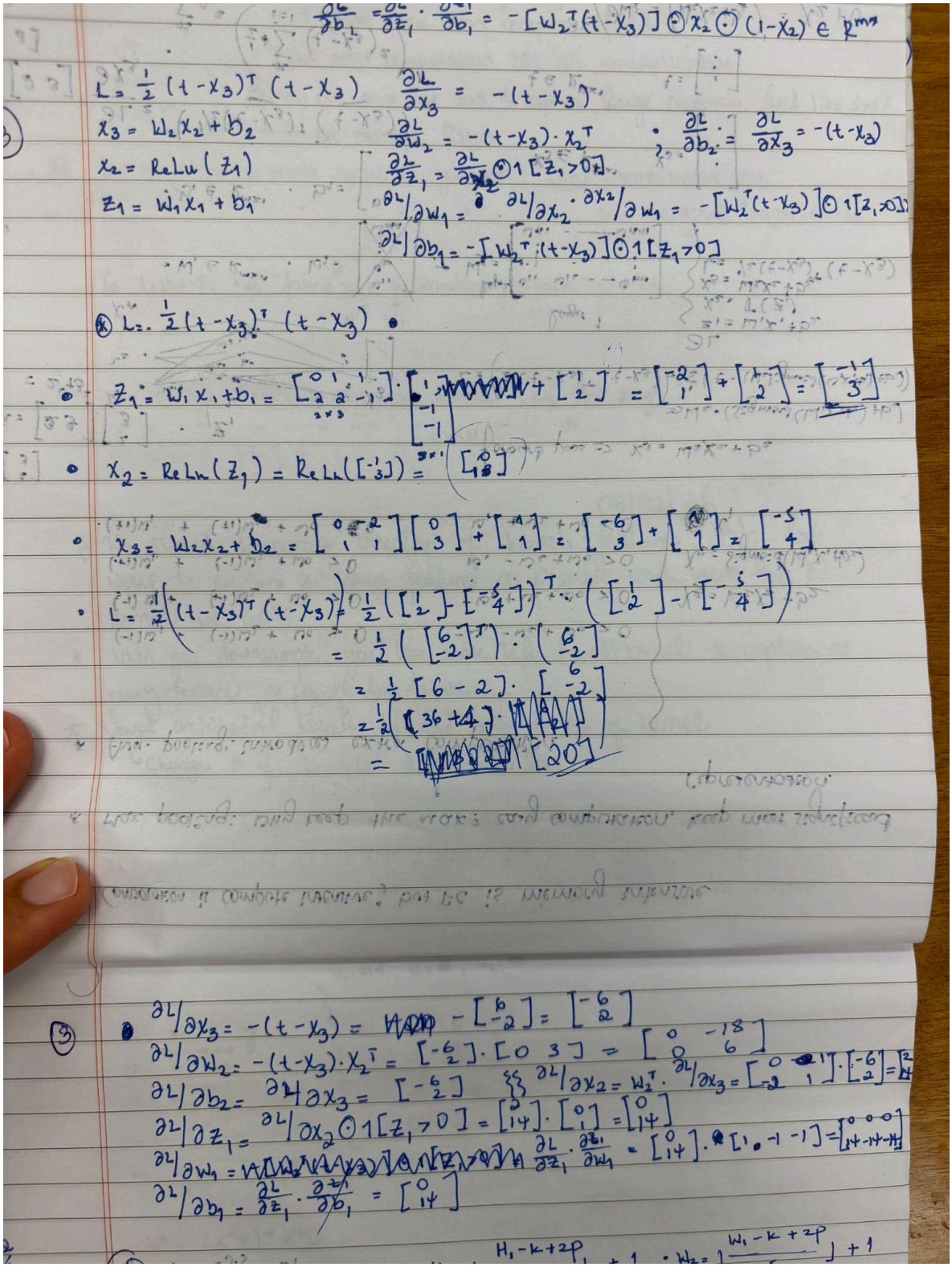
Problem 2.4

x1	**x2**	**s**	**y**
-1	-1	-1	-1
-1	+1	+1	+1
+1	-1	+1	+1
+1	+1	-1	-1

w20 = +1, w21 = +1, w22 = +1

Back Propagation (15 pts)

Problem 3.1 and Problem 3.2



2D Convolution
Problem 4.1

ece_661_hw1_4.ipynb

Problem 4.2

ece_661_hw1_4.ipynb

Lab: LMS Algorithm

SimpleNN.ipynb

Lab: Simple NN

SimpleNN.ipynb

w2 = [(w1 + 2p - k) / 2] + 1

If H * W = 32 * 32, P = 0 and S = 1
[(32 + 0 - 5) / 1] + 1: 28
[(32 + 0 - 10) / 1] + 1: 23

If H * W = 32 * 32, P = 2 and S = 1
[(32 + 4 - 5) / 1] + 1: 32
[(32 + 5 - 7) / 1] + 1: 31

4 2D Convolution (10 pts)

Problem 4.1 (5 pts)

In [2]:
```
import numpy as np
```

In [3]:
```
matrix = np.array([
    [0,  0, -1,  0, 0, 0, 1, 0, 0],
    [0, -1, -1, -1, 0, 1, 1, 1, 0],
    [-1, -1, -1, -1, 0, 1, 1, 1, 1],
    [0, -1, -1, -1, 0, 1, 1, 1, 0],
    [0,  0, -1,  0, 0, 0, 1, 0, 0]
])

kernel = np.array([
    [0, -1/2, 0],
    [-1/2, 2, -1/2],
    [0, -1/2, 0]
])

print(matrix)
print(kernel)

print("Shape:", matrix.shape)  # Outputs: (5, 9)
print("Shape: ", kernel.shape) # Outputs: (3, 3)
```

```
[[ 0  0 -1  0  0  0  1  0  0]
 [ 0 -1 -1 -1  0  1  1  1  0]
 [-1 -1 -1 -1  0  1  1  1  1]
 [ 0 -1 -1 -1  0  1  1  1  0]
 [ 0  0 -1  0  0  0  1  0  0]]
[[ 0.  -0.5  0. ]
 [-0.5  2.  -0.5]
 [ 0.  -0.5  0. ]]
Shape: (5, 9)
Shape:  (3, 3)
```

In [4]:
```
padded_matrix = np.pad(matrix, pad_width=1, mode='constant', constant_values=0)
padded_matrix
```

Out[4]:
```
array([[ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0],
       [ 0,  0,  0, -1,  0,  0,  0,  1,  0,  0,  0],
       [ 0,  0, -1, -1, -1,  0,  1,  1,  1,  0,  0],
       [ 0, -1, -1, -1, -1,  0,  1,  1,  1,  1,  0],
       [ 0,  0, -1, -1, -1,  0,  1,  1,  1,  0,  0],
       [ 0,  0,  0, -1,  0,  0,  0,  1,  0,  0,  0],
       [ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0]])
```

In [5]:
```
# Single convolution multiplication

hadamard_result = padded_matrix[:3, :3] * kernel
pixel_output = np.sum(hadamard_result)

print("Hadamard Matrix: ", hadamard_result)
print("Final pixel value: ", pixel_output)
```

```
Hadamard Matrix:  [[ 0. -0.  0.]
 [-0.  0. -0.]
 [ 0. -0. -0.]]
Final pixel value:  0.0
```

In [6]:
```
output = np.zeros((5, 9))

for i in range(0, 5):
    for j in range(0, 9):
        image_patch = padded_matrix[i:i+3, j:j+3]
        output[i, j] = np.sum(image_patch * kernel)

print("Final Convolved Output: ")
print(output)
```

```
Final Convolved Output:
[[ 0.   1.  -1.5  1.   0.  -1.   1.5 -1.   0. ]
 [ 1.  -1.   0.  -1.   0.   1.   0.   1.  -1. ]
 [-1.5  0.   0.  -0.5  0.   0.5  0.   0.   1.5]
 [ 1.  -1.   0.  -1.   0.   1.   0.   1.  -1. ]
 [ 0.   1.  -1.5  1.   0.  -1.   1.5 -1.   0. ]]
```

Problem 4.2 (5 pts)

In [7]:
```python
import numpy as np
from scipy.signal import convolve2d
from PIL import Image
import matplotlib.pyplot as plt


# Image Source: https://unsplash.com/photos/the-interior-of-a-car-NkBFPN9TSnQ
img = Image.open("hw1_img_001.jpg").convert("L")
arr = np.array(img, dtype=float)

kernel = np.array([[0, -0.5, 0],
                   [-0.5, 2, -0.5],
                   [0, -0.5, 0]])

out = convolve2d(arr, kernel, mode="same", boundary="fill", fillvalue=0)
out_clipped = np.clip(out, 0, 255).astype(np.uint8)

# Display original and processed images side-by-side
fig, axes = plt.subplots(1, 2, figsize=(10, 5))

axes[0].imshow(arr, cmap='gray')
axes[0].set_title('Original Image')
axes[0].axis('off')

axes[1].imshow(out_clipped, cmap='gray')
axes[1].set_title('Edge Detected (Convolved)')
axes[1].axis('off')

plt.show()
```



5. Lab: LMS Algorithm (15 pts)

(a) (3pt) Directly compute the least square (Wiener) solution with the provided dataset. What is the optimal weight W∗? What is the MSE loss of the whole dataset when the weight is set to W∗?

In [8]:
```
import numpy as np
import scipy.io as sio

mat_contents = sio.loadmat('dataset.mat')
print(sorted(mat_contents.keys()))

my_D = mat_contents['D']
```

```
my_X = mat_contents['X']

W = np.linalg.solve(my_X.T @ my_X, my_X.T @ my_D)

K = my_X.shape[0]
y = my_X @ W
e = my_D - y
J = (1/(2*K)) * (e.T @ e)

print("Optimal W: ", W)
print("MSE Loss: ", J.item())
```

```
['D', 'X', '__globals__', '__header__', '__version__']
Optimal W:  [[ 1.0006781 ]
 [ 1.00061145]
 [-2.00031968]]
MSE Loss:  5.0399515658683515e-05
```

(b) (4pt) Now consider that you can only train with 1 pair of data points and target each time. In such a case, the LMS algorithm should be used to find the optimal weight. Please initialize the weight vector as W0 = [−0.2, 0.0, 0.45]T , and update the weight with the LMS algorithm. After each epoch (every time you go through all the training data and loop back to the beginning), compute and record the MSE loss of the current weight on the whole dataset. Run LMS for 20 epochs with learning rate r = 0.005, report the weight you get in the end, and plot the MSE loss in log scale vs. Epochs.

In [9]:
```
import matplotlib.pyplot as plt

W0 = np.array([-0.2, 0.0, 0.45]).reshape(-1, 1)
current_W = W0.copy()

r = 0.005
K = my_X.shape[0]
J_history = []

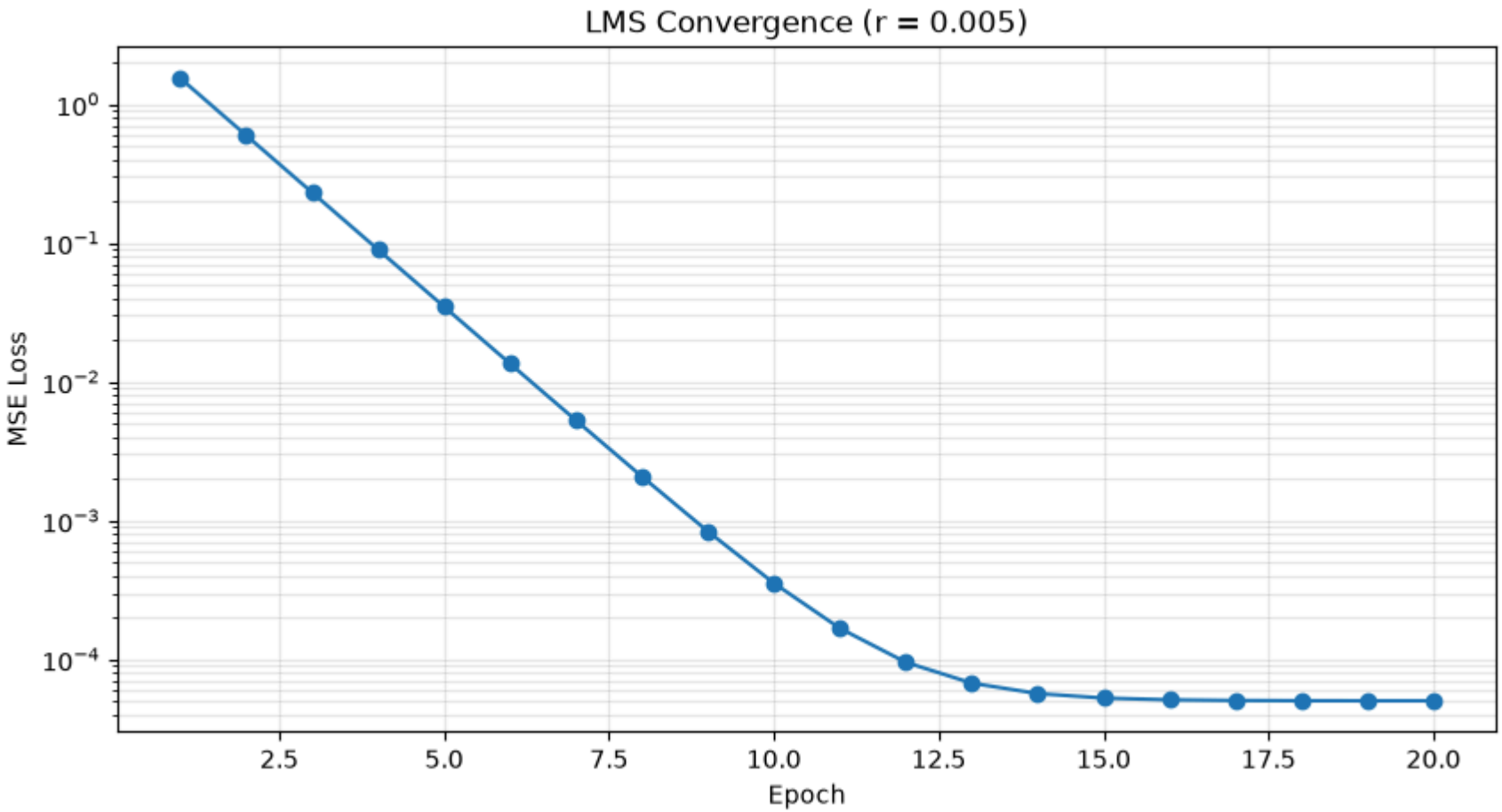
for epoch in range(20):
    for k in range(K):
        x_k = my_X[k:k+1, :].T
        d_k = my_D[k, 0]
        e_k = d_k - (x_k.T @ current_W).item()
        current_W = current_W + r * e_k * x_k
```

```
    y = my_X @ current_W
    e = my_D - y
    J = (1/(2*K)) * (e.T @ e)
    J_history.append(J.item())

print("Final W after 20 epochs:", current_W)

plt.figure(figsize=(10, 5))
plt.plot(range(1, 21), J_history, marker='o')
plt.yscale('log')
plt.xlabel('Epoch')
plt.ylabel('MSE Loss')
plt.title('LMS Convergence (r = 0.005)')
plt.grid(True, which='both', alpha=0.3)
plt.show()
```

```
Final W after 20 epochs: [[ 1.00066744]
 [ 1.0005943 ]
 [-2.00030134]]
```



(3pt) Scatter plot the points (x1k, x2k, dk) for all 100 data-target pairs in a 3D figureb, and plot the lines corresponding to the linear models you got in (a) and (b) respectively in the same figure. Observe if the linear models fit the data well.

In [10]:
```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from matplotlib.lines import Line2D


x1 = my_X[:, 1]
x2 = my_X[:, 2]
```

```
d  = my_D[:, 0]

N = 25
x1_range = np.linspace(x1.min(), x1.max(), N)
x2_range = np.linspace(x2.min(), x2.max(), N)
X1_grid, X2_grid = np.meshgrid(x1_range, x2_range)

w0_a, w1_a, w2_a = W.flatten()            # part (a) weight
D_plane_a = w0_a + w1_a * X1_grid + w2_a * X2_grid

w0_b, w1_b, w2_b = current_W.flatten()    # part (b) weight
D_plane_b = w0_b + w1_b * X1_grid + w2_b * X2_grid

fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

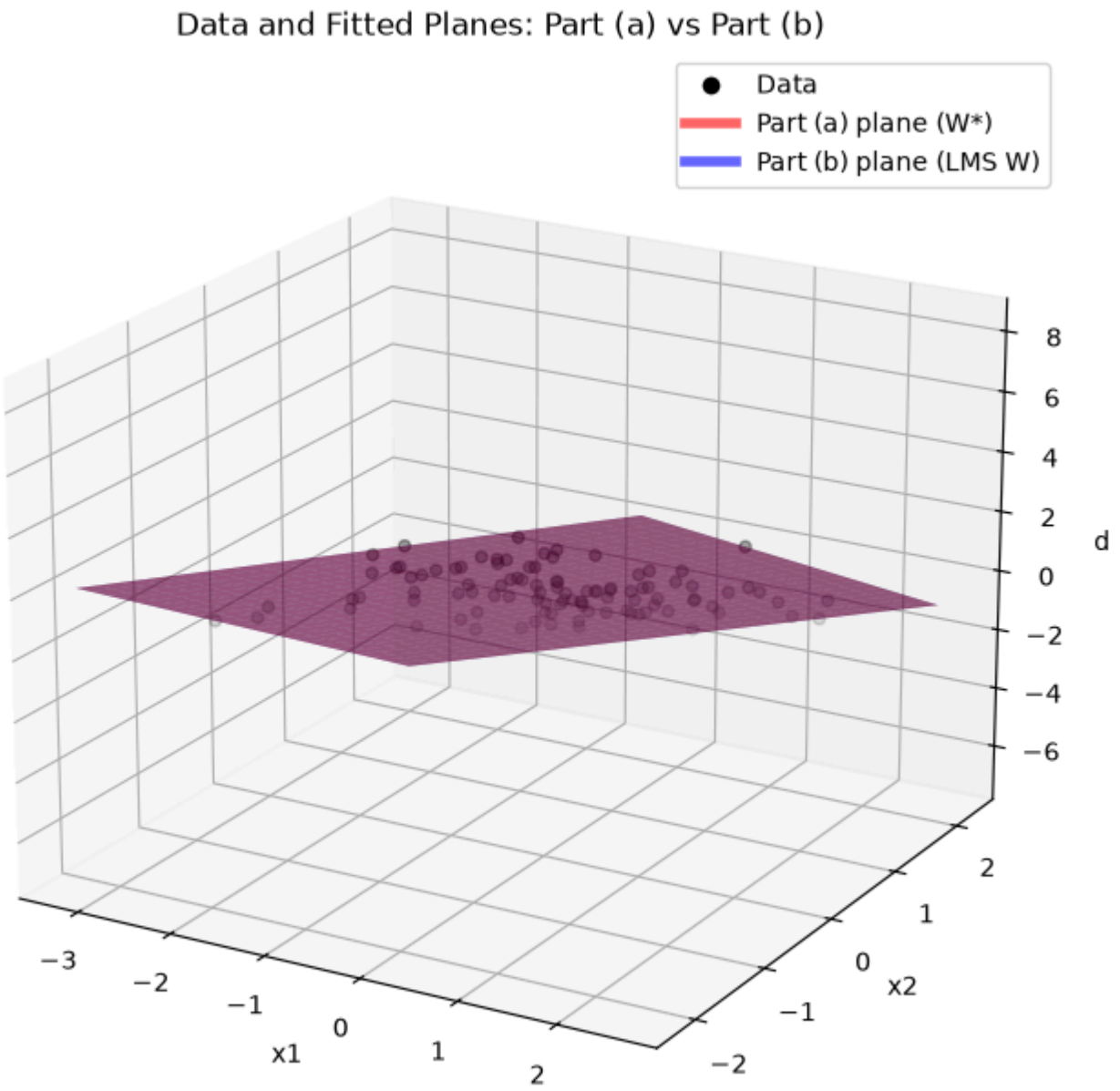
ax.scatter(x1, x2, d, color='black', alpha=0.6)
ax.plot_surface(X1_grid, X2_grid, D_plane_a, alpha=0.4, color='red')
ax.plot_surface(X1_grid, X2_grid, D_plane_b, alpha=0.4, color='blue')

ax.view_init(elev=20, azim=-60)

ax.set_xlabel('x1')
ax.set_ylabel('x2')
ax.set_zlabel('d')
ax.set_title('Data and Fitted Planes: Part (a) vs Part (b)')

legend_elements = [
    Line2D([0], [0], marker='o', color='w', markerfacecolor='black', markersize=8, label='Data'),
    Line2D([0], [0], color='red', lw=4, alpha=0.6, label='Part (a) plane (W*)'),
    Line2D([0], [0], color='blue', lw=4, alpha=0.6, label='Part (b) plane (LMS W)'),
]
ax.legend(handles=legend_elements)

plt.show()
```


Data and Fitted Planes: Part (a) vs Part (b)
Data
Part (a) plane (W*)
Part (b) plane (LMS W)
8
6
4
2
0
−2
−4
−6
d
−3
−2
−1
0
1
2
x1
2
1
0
−1
−2
x2

(d) (5pt) Learning rate r is an important hyperparameter for the LMS algorithm, as well as for CNN optimization. Here, try repeat the process in (b) with r set to 0.01, 0.05, 0.1 and 0.5 respectively. Together with the result you got in (b), plot the MSE losses of all sets of experiments in log scale vs. Epochs in one figure. Then try further enlarge the learning rate to r = 1 and observe how the MSE changes. Based on these observations, comments on how the learning rate affects the speed and quality of the learning process. (Note: The learning rate tuning for the CNN optimization will be introduced in Lecture 7.)

In [11]:
```
def learning_rate_weight_update(W_init, r):
    current_W = W_init.copy()
    J_history = []
    K = my_X.shape[0]

    for epoch in range(20):
        for k in range(K):
            x_k = my_X[k:k+1, :].T
            d_k = my_D[k, 0]
            e_k = d_k - (x_k.T @ current_W).item()
            current_W = current_W + r * e_k * x_k

        y = my_X @ current_W
        e = my_D - y
        J = (1/(2*K)) * (e.T @ e)
        J_history.append(J.item())

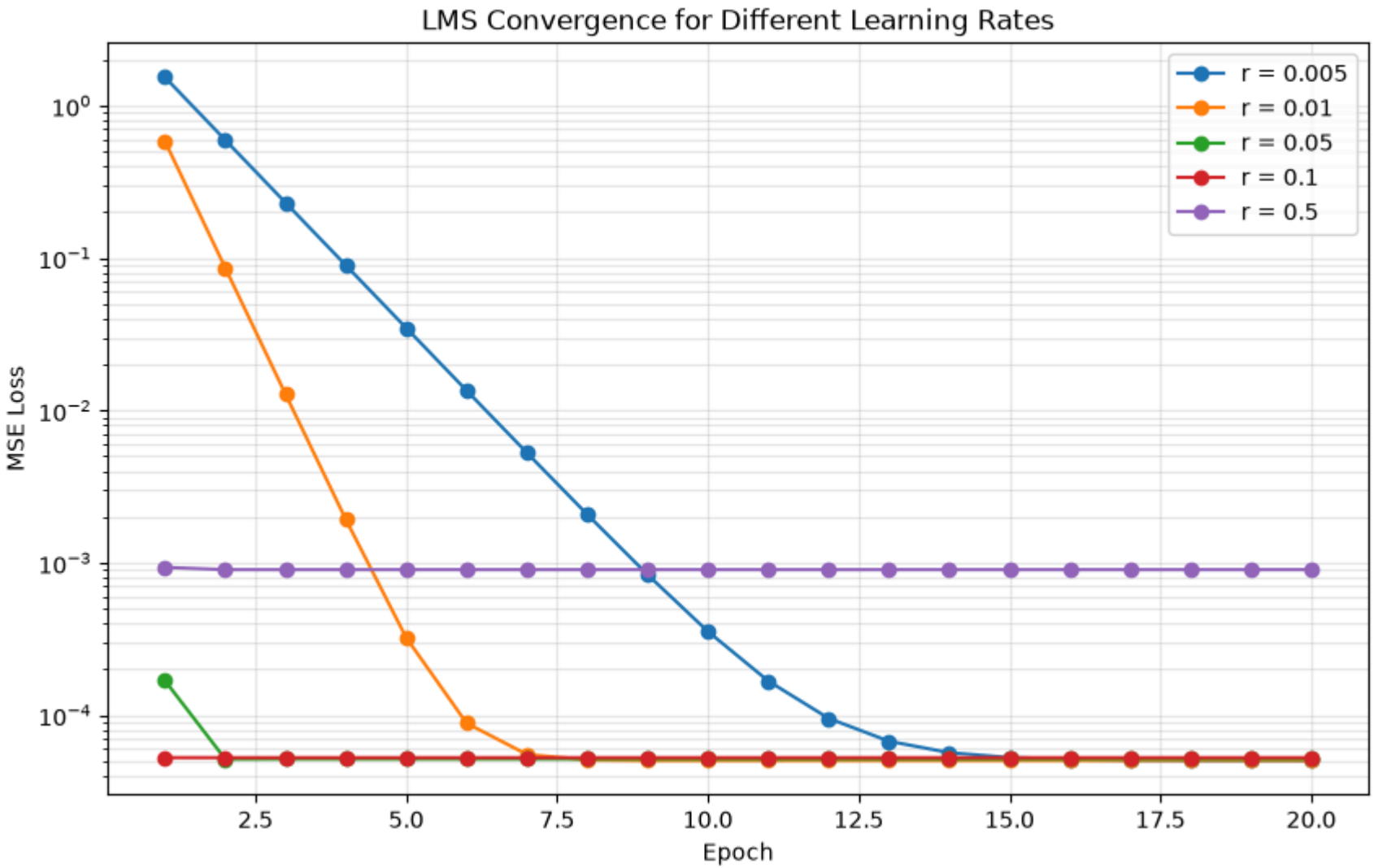
    return J_history, current_W


W0 = np.array([-0.2, 0.0, 0.45]).reshape(-1, 1)
lst_learning_rate = [0.005, 0.01, 0.05, 0.1, 0.5]

results = {}   # r -> (J_history, final_W)
for r in lst_learning_rate:
    J_hist, final_W = learning_rate_weight_update(W0, r)
    results[r] = (J_hist, final_W)

plt.figure(figsize=(10, 6))
for r in lst_learning_rate:
    J_hist, _ = results[r]
    plt.plot(range(1, 21), J_hist, marker='o', label=f'r = {r}')
```

```
plt.yscale('log')
plt.xlabel('Epoch')
plt.ylabel('MSE Loss')
plt.title('LMS Convergence for Different Learning Rates')
plt.legend()
plt.grid(True, which='both', alpha=0.3)
plt.show()
```



In [12]:
```
J_hist_r1, final_W_r1 = learning_rate_weight_update(W0, 1)

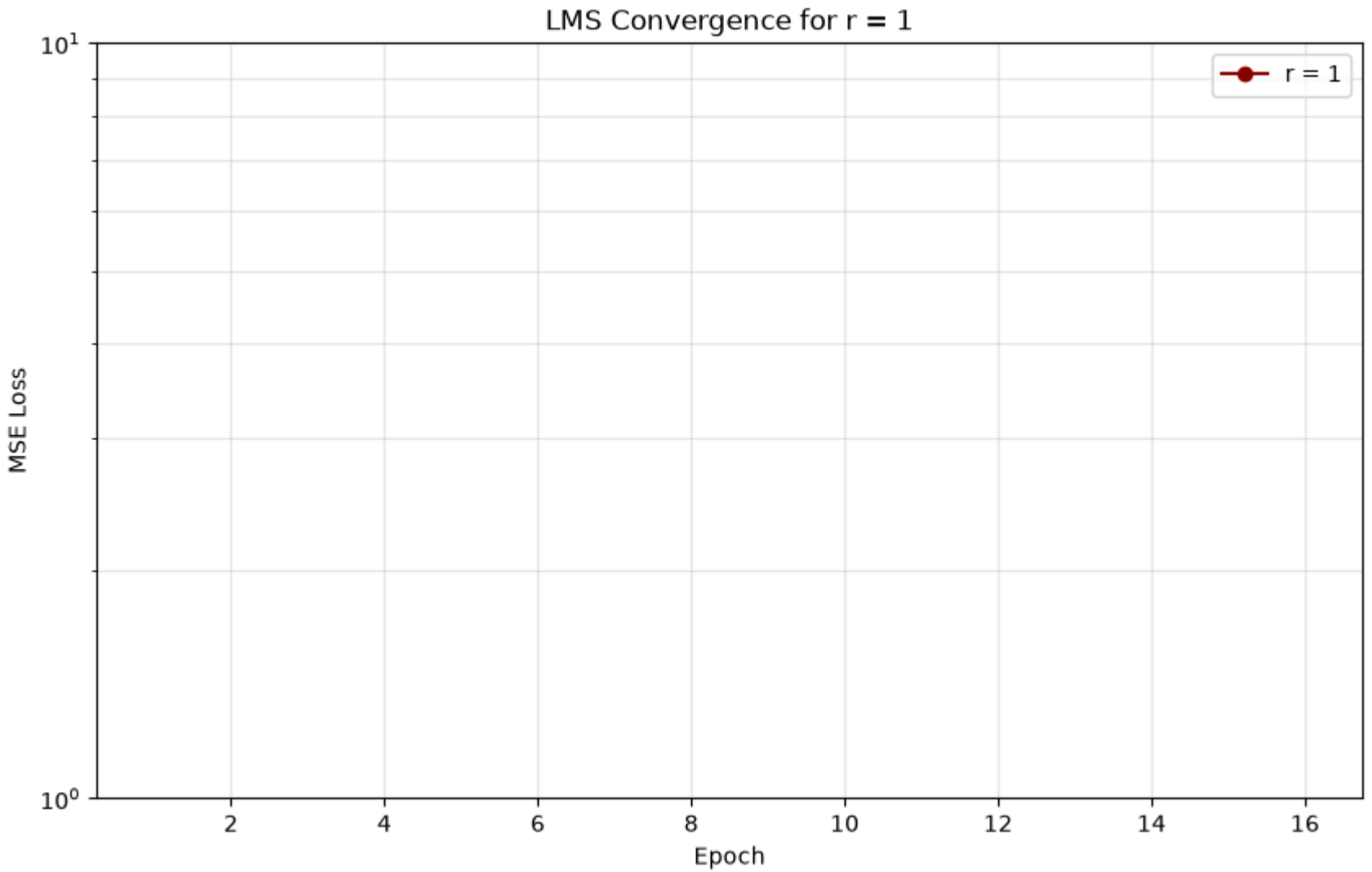
plt.figure(figsize=(10, 6))
plt.plot(range(1, 21), J_hist_r1, marker='o', color='darkred', label='r = 1')

plt.yscale('log')
plt.xlabel('Epoch')
```

```
plt.ylabel('MSE Loss')
plt.title('LMS Convergence for r = 1')
plt.legend()
plt.grid(True, which='both', alpha=0.3)
plt.show()

print("Final W after 20 epochs (r = 1):", final_W_r1)
print("J history (r = 1):", J_hist_r1)
```

```
/Users/temesmacbookair/Development/ece661_ce_ml_nn/.venv/lib/python3.14/site-packages/matplotlib/scale.py:375: Runti
meWarning: overflow encountered in exp
  return np.exp(values * np.log(self.base))
```




```
Final W after 20 epochs (r = 1): [[ 1.28471518e+188]
 [-4.10303918e+187]
 [-1.33708699e+188]]
J history (r = 1): [6.330673253512807e+18, 4.1084066028668144e+37, 2.6662243288047043e+56, 1.73029421346673e+75, 1.1
22905538296755e+94, 7.287297374769916e+112, 4.729222647603912e+131, 3.0691140625114774e+150, 1.991756749595622e+16
9, 1.292586351878208e+188, 8.388471520937628e+206, 5.443849407456229e+225, 3.53288394638893e+244, 2.292728553724759e
+263, 1.4879074152542656e+282, 9.656042677934965e+300, inf, inf, inf, inf]
```

Answer: A bigger learning rate makes the model learn faster. 0.05 and 0.1 reach their best result in just one pass through the data, but a small rate like 0.005 takes over 15 passes to get there. But if the rate gets too big, like 0.5, the model starts overshooting the target slightly each time, so it ends up a bit less accurate even though it's fast. At r=1, the overshooting becomes so extreme that the model gets worse and worse instead of better.

Code block 1: Package initialization

Import required packages, do not change.

In [2]:
```
import argparse
import os, sys
import time
import datetime
import math
import numpy as np
import matplotlib.pyplot as plt

# Import pytorch dependencies
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.nn.modules.utils import _single, _pair, _triple
```

Code block 2: Useful classes

Customized implementation of nn.Conv2d and nn.Linear, do not change.

In [3]:
```
class CONV(nn.Conv2d):
    def __init__(self, in_channels, out_channels, kernel_size, stride=1,
                 padding=0, dilation=1, groups=1,
                 bias=False, padding_mode='zeros'):
        self.input = None
        self.output = None
        kernel_size = _pair(kernel_size)
        stride = _pair(stride)
        padding = _pair(padding)
        dilation = _pair(dilation)
        super(CONV, self).__init__(
            in_channels, out_channels, kernel_size, stride, padding, dilation,
            groups, bias, padding_mode)
```

```
    def forward(self, input):
        self.input = input
        # 'pytorch 2.0'
        self.output = super().forward(input)
        return self.output

class FC(nn.Linear):
    def __init__(self, in_features, out_features, bias=True):
        self.input = None
        self.output = None
        super(FC, self).__init__(in_features, out_features, bias)

    def forward(self, input):
        self.input = input
        self.output = F.linear(input, self.weight, self.bias)
        return self.output
```

Lab 2

Code block 3: SimpleNN implementation

Please follow the instructions in Lab 2(a) and fill in the code in the lines marked **Your code here**

In [4]:
```
"""
Lab 2(a)
Build the SimpleNN model by following Table 1
"""

# Create the neural network module: LeNet-5
class SimpleNN(nn.Module):
    def __init__(self):
        super(SimpleNN, self).__init__()
        # Layer definition
        self.conv1 =     CONV(in_channels=3, out_channels=16, kernel_size=5, padding=2, stride=1)
        self.conv2 =     CONV(in_channels=16, out_channels=16, kernel_size=3, padding=2, stride=1)
        self.conv3 =     CONV(in_channels=16, out_channels=24, kernel_size=7, padding=2, stride=1)
        self.fc1   =     FC(in_features=216, out_features=24)
```

```
        self.fc2   =     FC(in_features=24, out_features=10)

    def forward(self, x):
        # Forward pass computation
        # Conv 1
        x = F.relu(self.conv1(x))
        # MaxPool
        x = F.max_pool2d(x, kernel_size=4, stride=2, padding=0)
        # Conv 2
        x = F.relu(self.conv2(x))
        # MaxPool
        x = F.max_pool2d(x, kernel_size=3, stride=2, padding=0)
        # Conv 3
        x = F.relu(self.conv3(x))
        # MaxPool
        x = F.max_pool2d(x, kernel_size=2, stride=2, padding=0)
        # Flatten
        x = x.reshape(x.size(0), -1)
        # FC 1
        x = F.relu(self.fc1(x))
        # FC 2
        x = F.relu(self.fc2(x))
        out = x
        return out

# GPU check
device = 'cuda' if torch.cuda.is_available() else 'cpu'
if device =='cuda':
    print("Run on GPU...")
else:
    print("Run on CPU...")

# Model Definition
net = SimpleNN()
net = net.to(device)

# Test forward pass
data = torch.randn(5,3,32,32)
data = data.to(device)
# Forward pass "data" through "net" to get output "out"
out =     net(data)
```

```
# Check output shape
assert(out.detach().cpu().numpy().shape == (5,10))
print("Forward pass successful")
```

```
Run on CPU...
Forward pass successful
```

Code block 4: Shape observation

Please follow the instructions in Lab 2(a) and fill in the code in the lines marked **Your code here**. Gather the printed results in Table 2 in your report.

In [5]:
```
"""
Lab 2(b)
"""
# Forward pass of a single image
data = torch.randn(1,3,32,32).to(device)
# Forward pass "data" through "net" to get output "out"
out = net(data)

# Iterate through all the CONV and FC layers of the model
for name, module in net.named_modules():
    if isinstance(module, CONV) or isinstance(module, FC):
        # Get the input feature map of the module as a NumPy array
        input = module.input.detach().cpu().numpy()
        # Get the output feature map of the module as a NumPy array
        output = module.output.detach().cpu().numpy()
        # Get the weight of the module as a NumPy array
        weight = module.weight.detach().cpu().numpy()
        # Compute the number of parameters in the weight
        num_Param = weight.size
        # Compute the number of MACs in the layer
        if isinstance(module, CONV):
            num_MAC = weight.size * output.shape[-1] * output.shape[-2]
        else:
            num_MAC = weight.size

        print(f'{name:10} {str(input.shape):20} {str(output.shape):20} {str(weight.shape):20} {str(num_Param):10} {
```

```
conv1      (1, 3, 32, 32)      (1, 16, 32, 32)     (16, 3, 5, 5)       1200       1228800
conv2      (1, 16, 15, 15)     (1, 16, 17, 17)     (16, 16, 3, 3)      2304       665856
conv3      (1, 16, 8, 8)       (1, 24, 6, 6)       (24, 16, 7, 7)      18816      677376
fc1        (1, 216)            (1, 24)             (24, 216)           5184       5184
fc2        (1, 24)             (1, 10)             (10, 24)            240        240
```

Layer	Input shape	Output shape	Weight shape	# Param	# MAC
Conv 1	(1, 3, 32, 32)	(1, 16, 32, 32)	(16, 3, 5, 5)	1,200	1,228,800
Conv 2	(1, 16, 15, 15)	(1, 16, 17, 17)	(16, 16, 3, 3)	2,304	665,856
Conv 3	(1, 16, 8, 8)	(1, 24, 6, 6)	(24, 16, 7, 7)	18,816	677,376
FC1	(1, 216)	(1, 24)	(24, 216)	5,184	5,184
FC2	(1, 24)	(1, 10)	(10, 24)	240	240

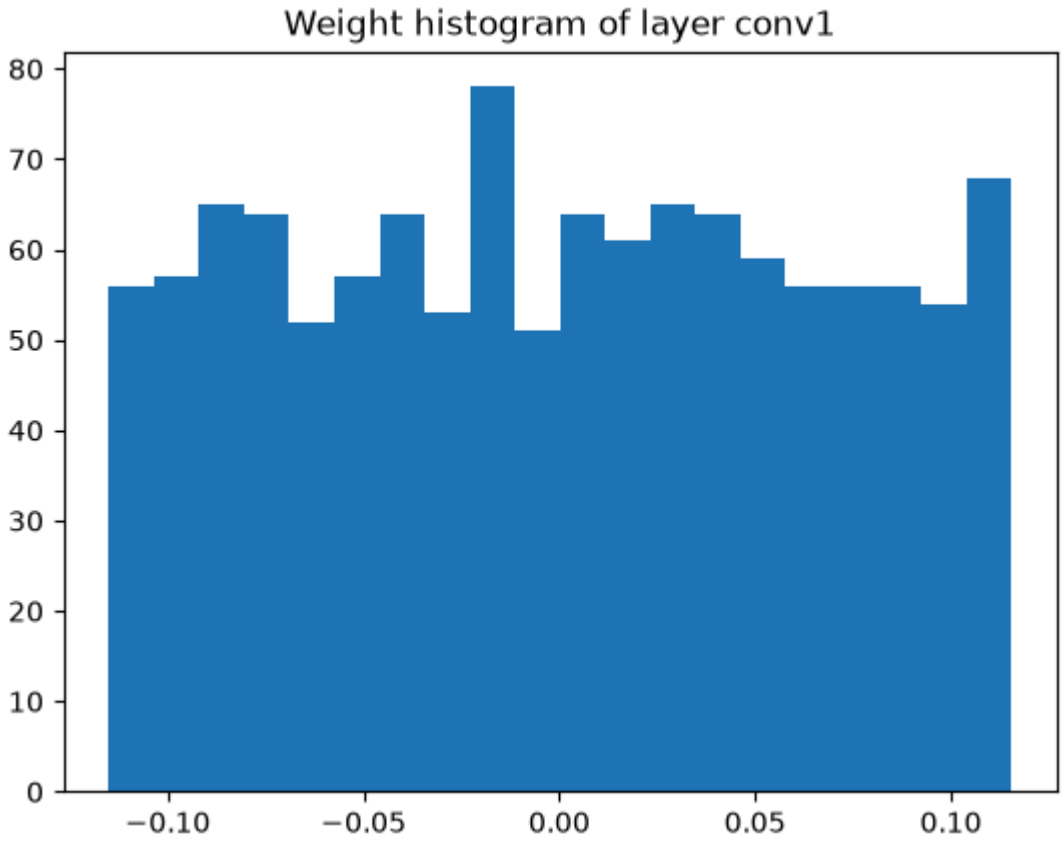
Lab 3 (Bonus)

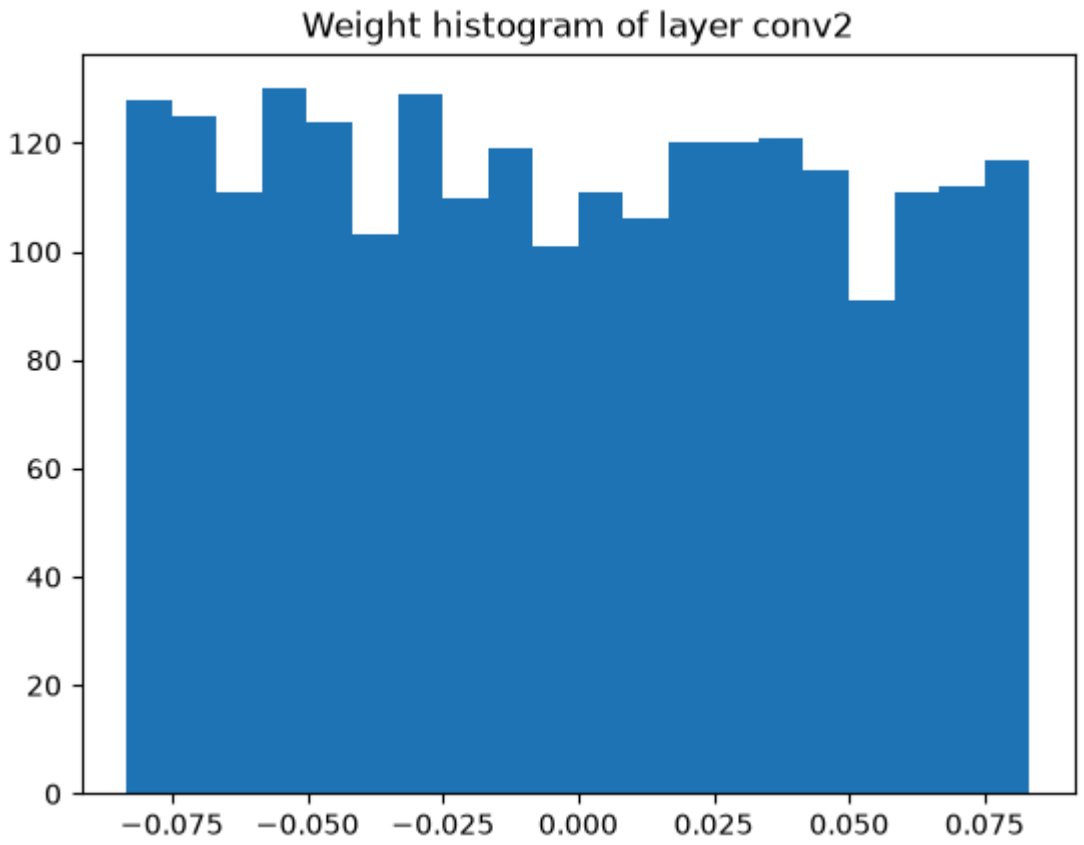
Code block 5: Initial weight histogram

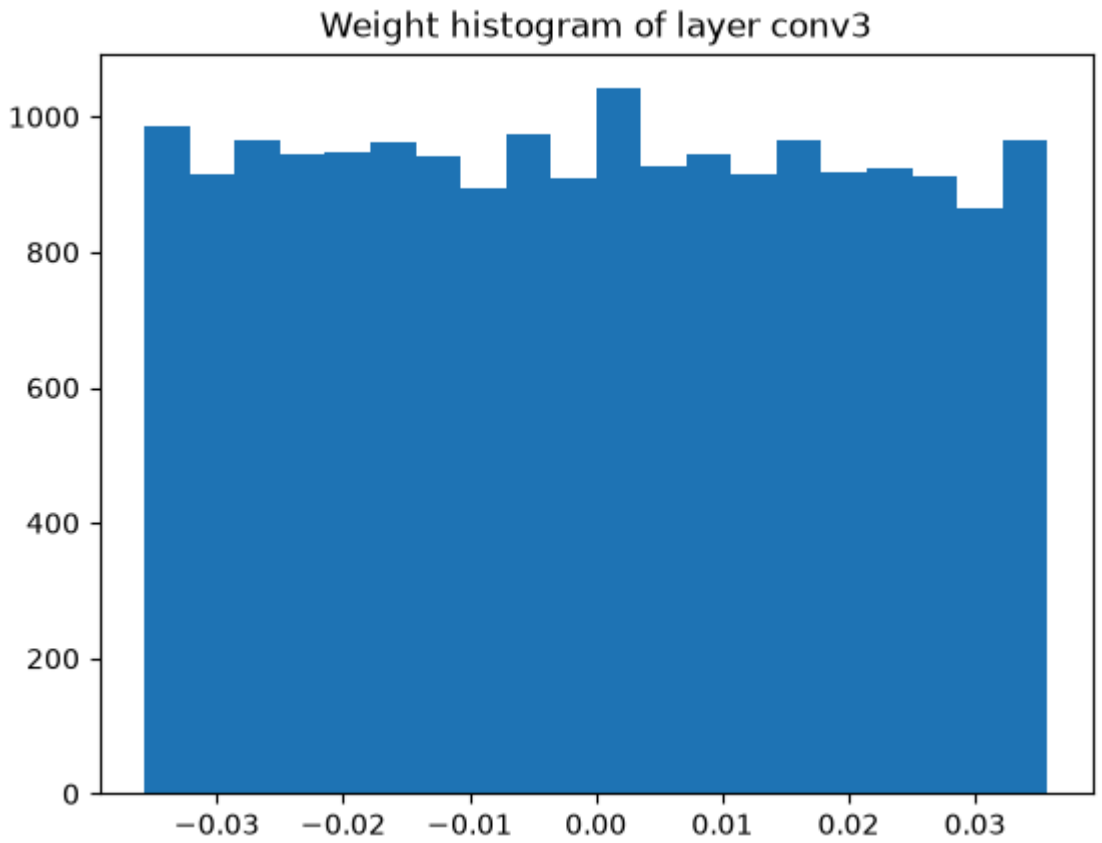
Please follow the instructions in Lab 3(a) and fill in the code in the lines marked **Your code here**. Copy the output figures into your report.

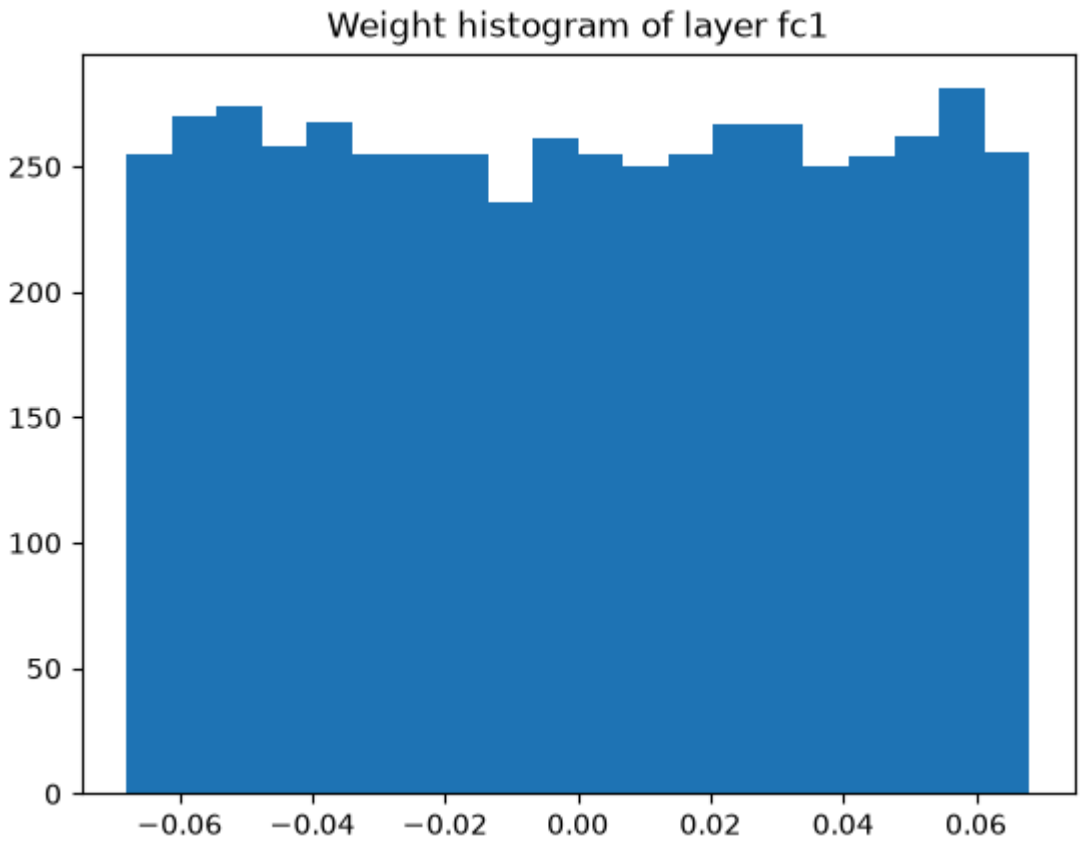
In [7]:
```
"""
Lab 3(a)
"""
for name, module in net.named_modules():
    if isinstance(module, CONV) or isinstance(module, FC):
        # Get the weight of the module as a NumPy array
        weight = module.weight.detach().cpu().numpy()

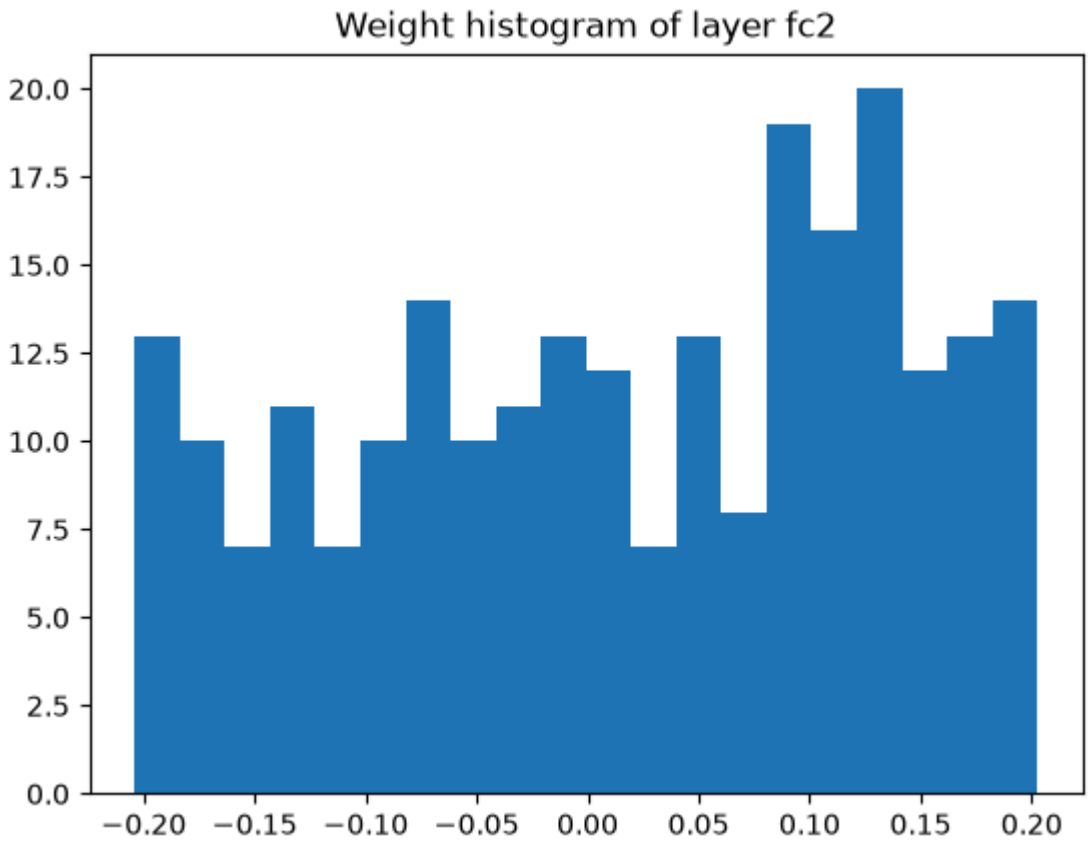
        # Reshape for histogram
        weight = weight.reshape(-1)
        _ = plt.hist(weight, bins=20)
        plt.title("Weight histogram of layer "+name)
        plt.show()
```


Weight histogram of layer conv1
80
70
60
50
40
30
20
10
0
−0.10
−0.05
0.00
0.05
0.10


Weight histogram of layer conv2
120
100
80
60
40
20
0
−0.075
−0.050
−0.025
0.000
0.025
0.050
0.075


Weight histogram of layer conv3
1000
800
600
400
200
0
−0.03
−0.02
−0.01
0.00
0.01
0.02
0.03


Weight histogram of layer fc1
250
200
150
100
50
0
−0.06
−0.04
−0.02
0.00
0.02
0.04
0.06



Code block 6: Gradient histogram

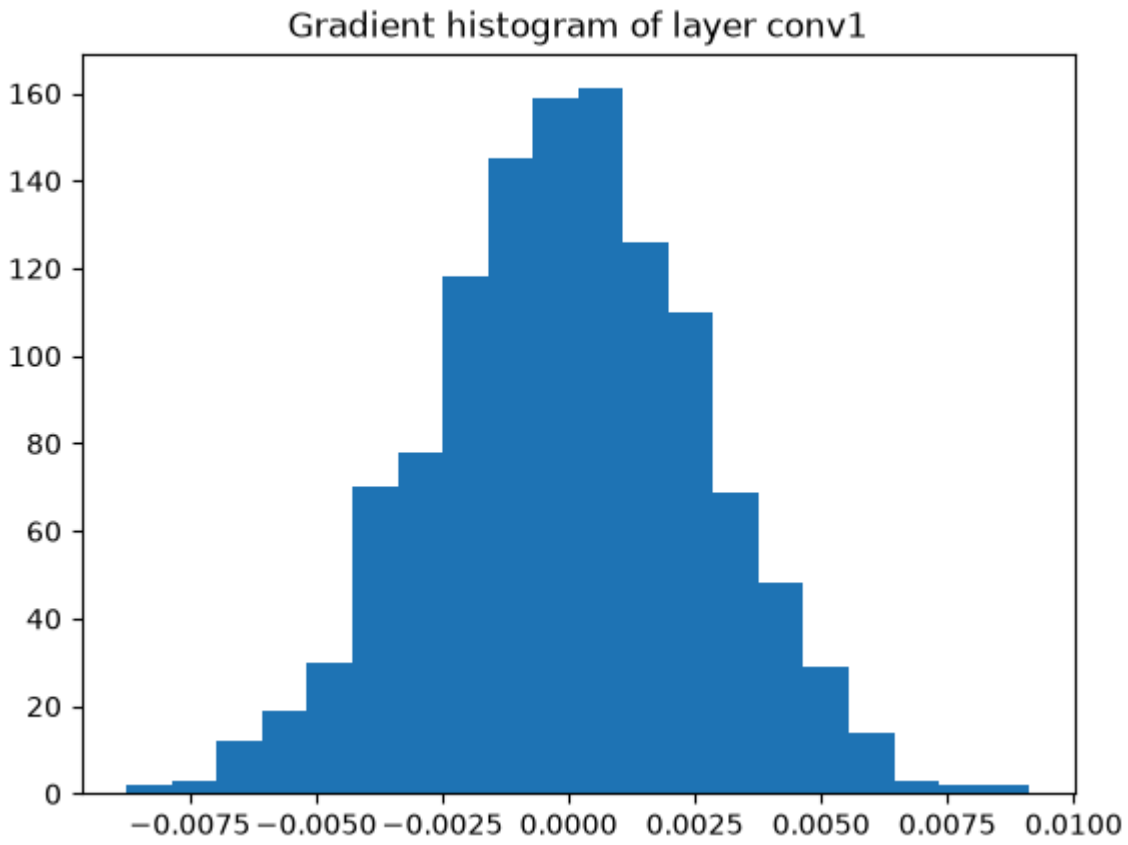
Please follow the instructions in Lab 3(b) and fill in the code in the lines marked **Your code here**. Copy the output figures into your report.

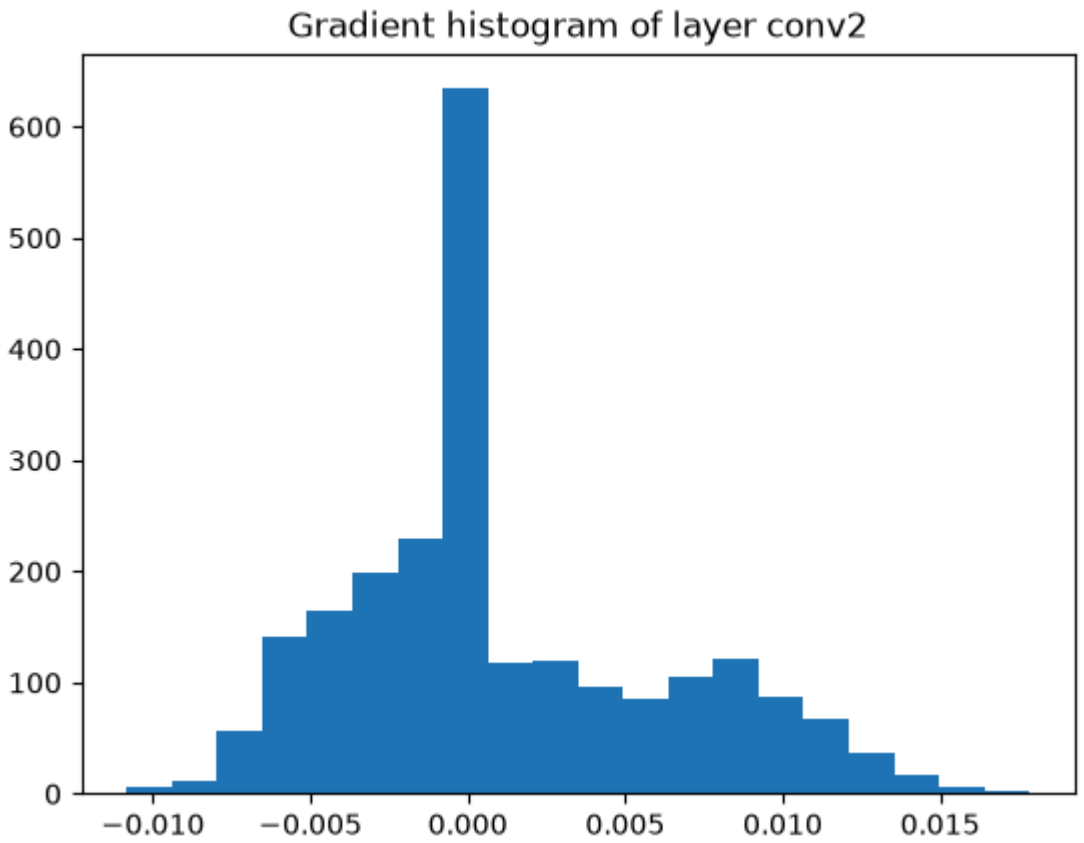
In [9]:
```
'''
Lab 3(b)
'''
# Loss definition
criterion = nn.MSELoss()
# Random target
target = torch.randn(1, 10).to(device)
```

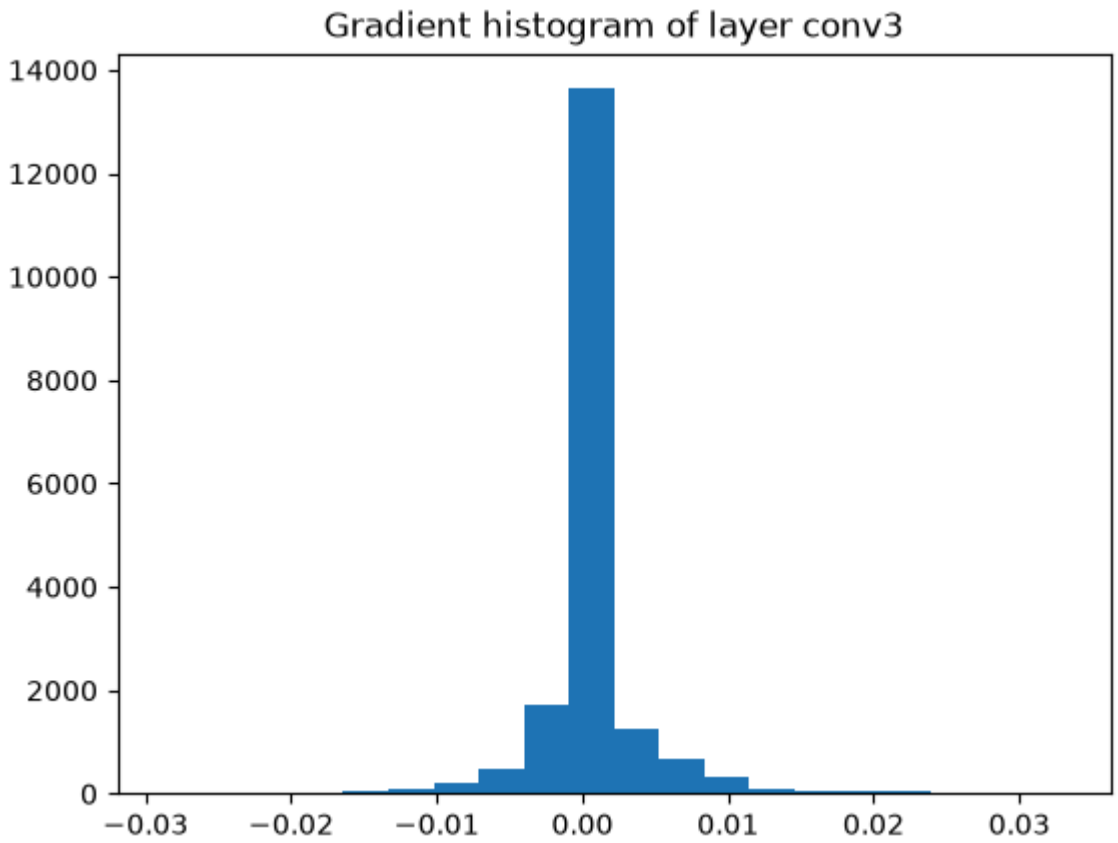
```
# Loss computation
loss = loss = criterion(out, target)
# Backward pass for gradients
net.zero_grad()
loss.backward()

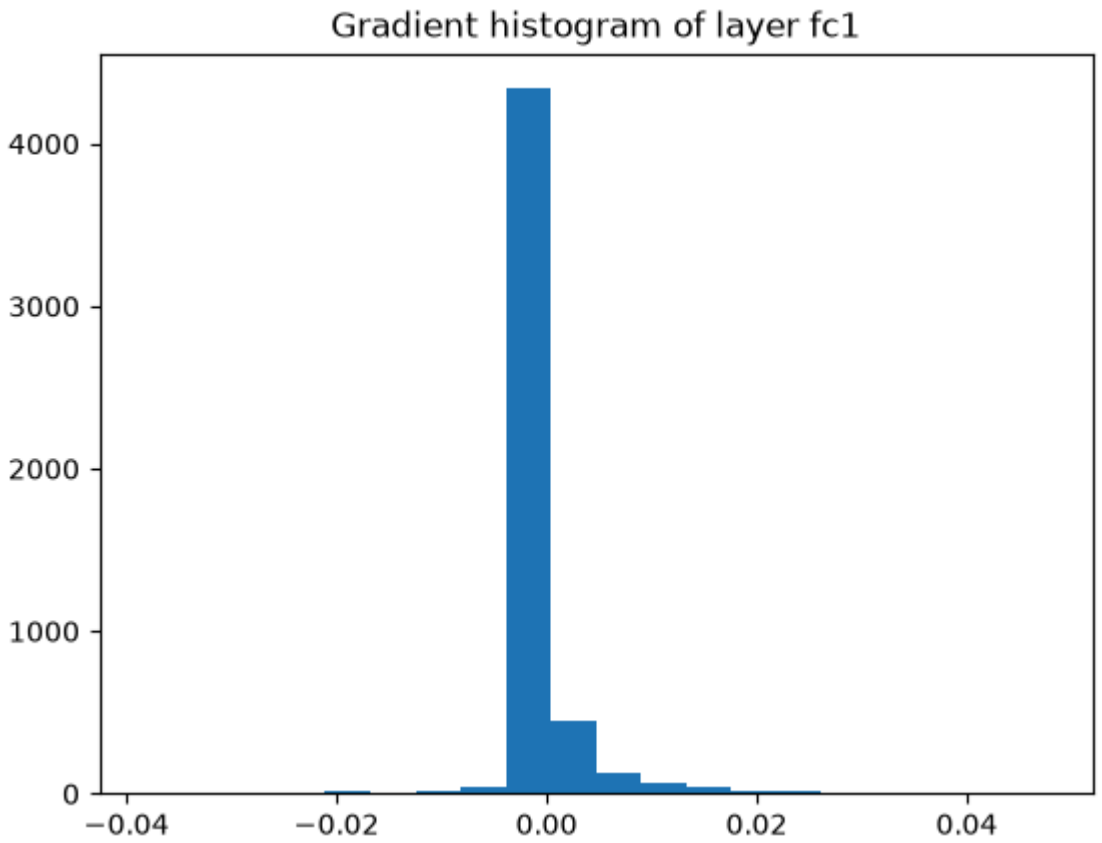
for name, module in net.named_modules():
    if isinstance(module, CONV) or isinstance(module, FC):
        # Get the gradient of the module as a NumPy array
        gradient = gradient = module.weight.grad.detach().cpu().numpy()

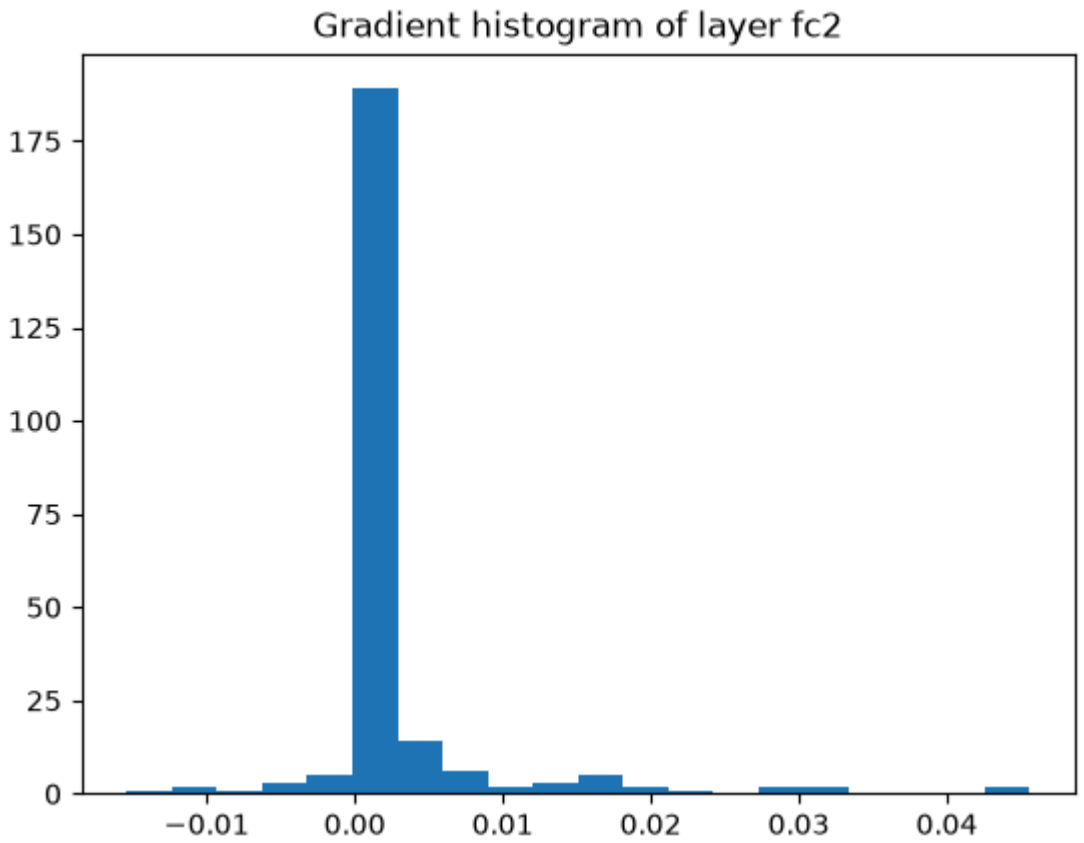
        # Reshape for histogram
        gradient = gradient.reshape(-1)
        _ = plt.hist(gradient, bins=20)
        plt.title("Gradient histogram of layer "+name)
        plt.show()
```


Gradient histogram of layer conv1
160
140
120
100
80
60
40
20
0
−0.0075
−0.0050
−0.0025
0.0000
0.0025
0.0050
0.0075
0.0100


Gradient histogram of layer conv2
600
500
400
300
200
100
0
−0.010
−0.005
0.000
0.005
0.010
0.015


Gradient histogram of layer conv3
14000
12000
10000
8000
6000
4000
2000
0
−0.03
−0.02
−0.01
0.00
0.01
0.02
0.03


Gradient histogram of layer fc1
4000
3000
2000
1000
0
−0.04
−0.02
0.00
0.02
0.04



Code block 7: Zero initialization?

Please follow the instructions in Lab 3(c) and fill in the code in the lines marked **Your code here**. Copy the output figures into your report.

In [10]:
```
'''
Lab 3(c)
'''
# Set model weights to zero
for name, module in net.named_modules():
    if isinstance(module, CONV) or isinstance(module, FC):
        # Set the weight of each module to all zero
```

```
        module.weight.data.fill_(0)

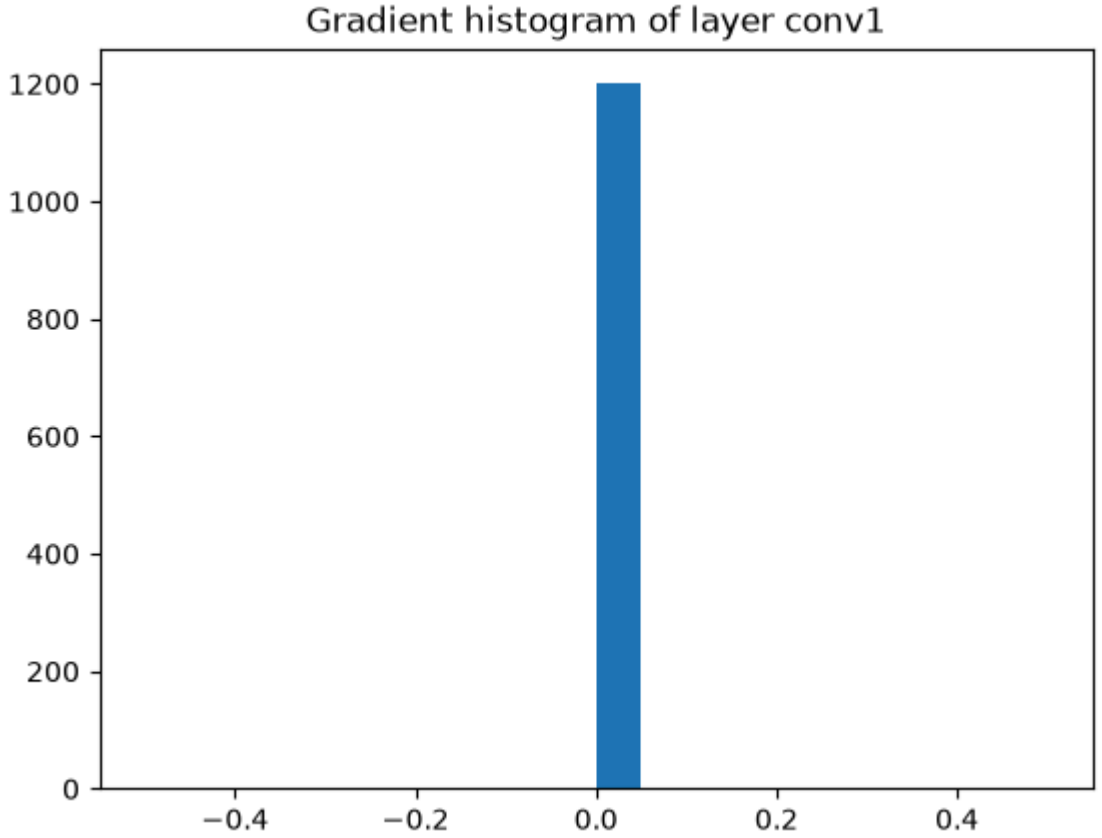
# Reset gradients
net.zero_grad()

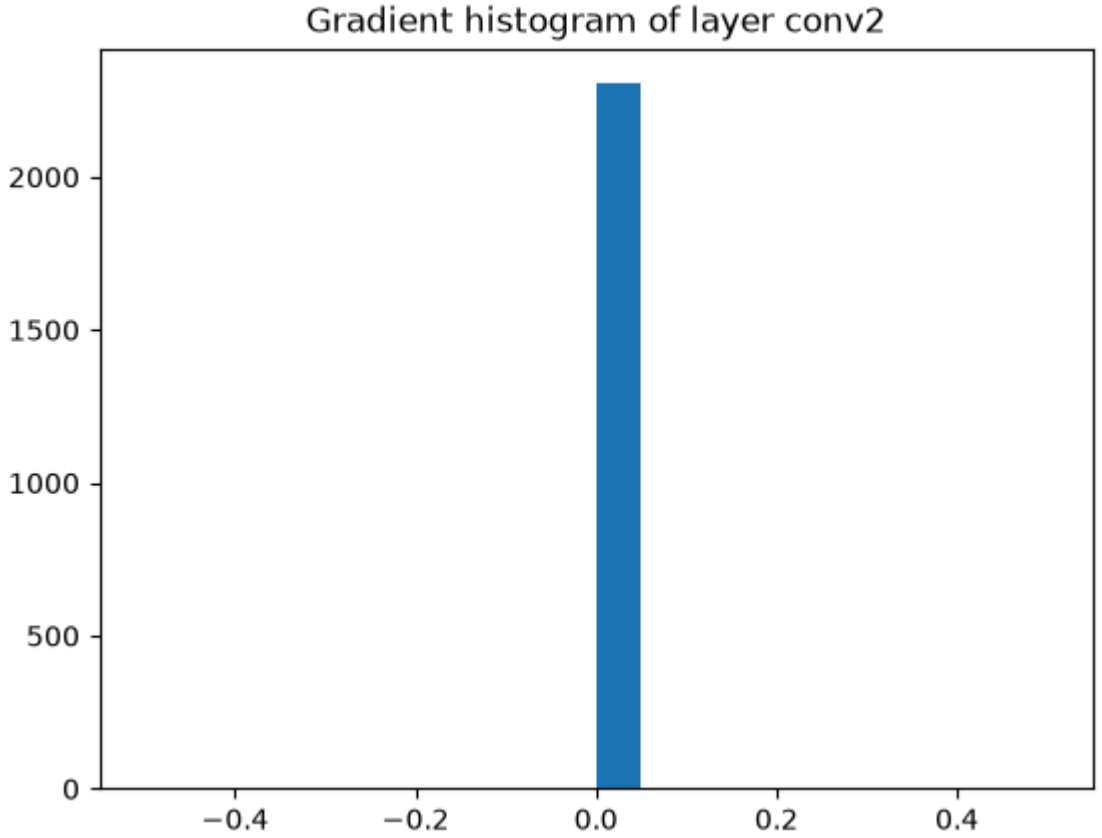
# Forward and backward pass
# Random data and target
data = torch.randn(1,3,32,32).to(device)
target = torch.randn(1, 10).to(device)

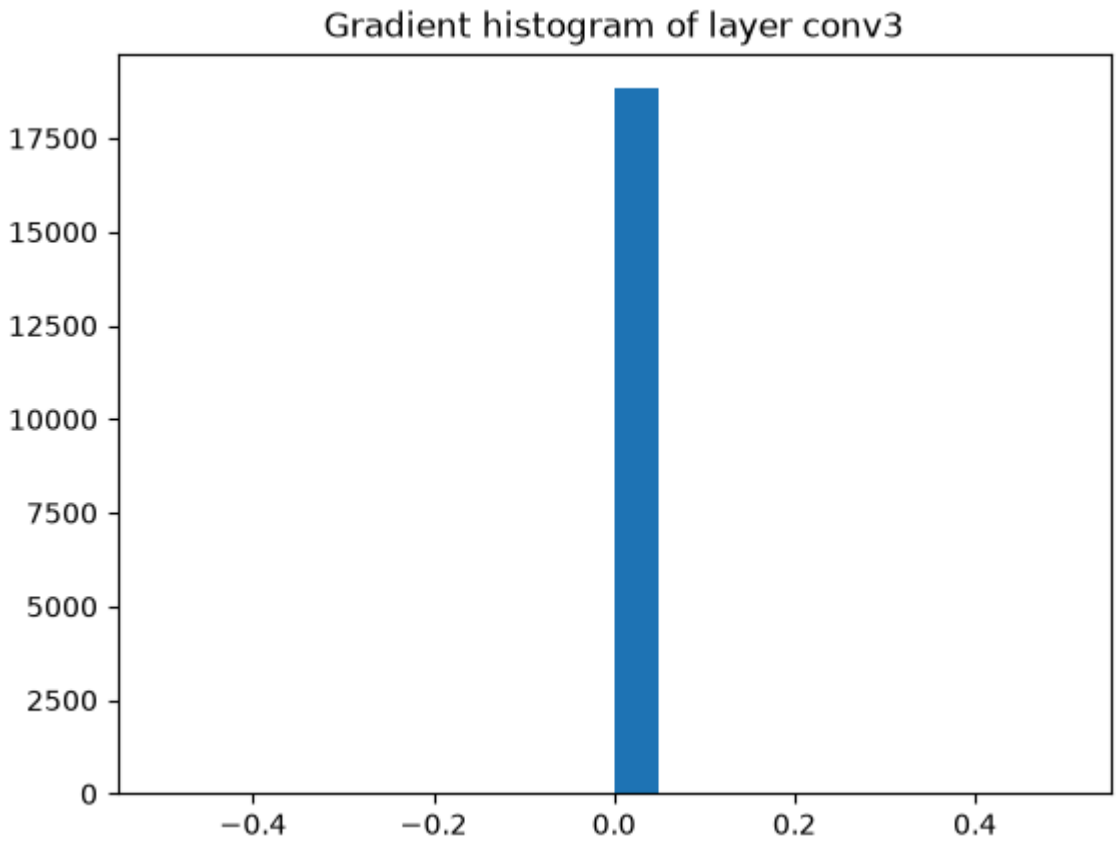
# Forward pass
out = net(data)
# Loss computation
loss = criterion(out, target)
# Backward pass
loss.backward()

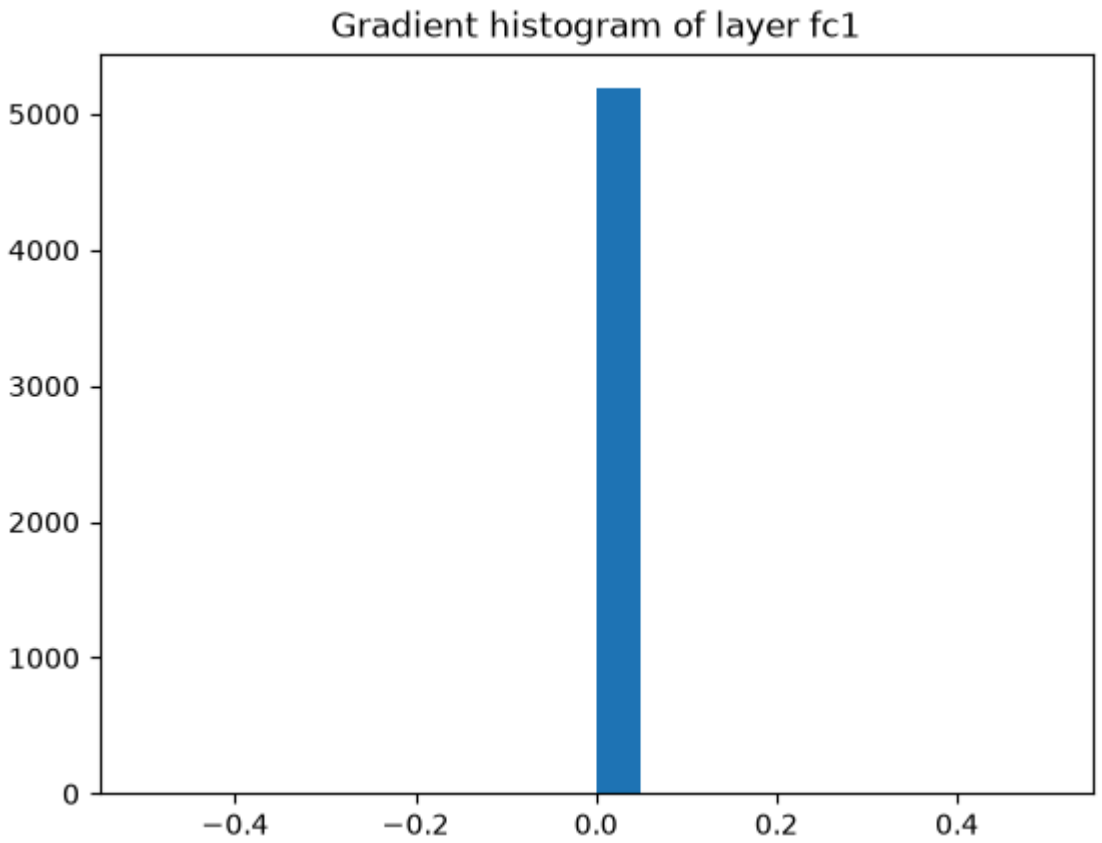
for name, module in net.named_modules():
    if isinstance(module, CONV) or isinstance(module, FC):
        # Get the gradient of the module as a NumPy array
        gradient = module.weight.grad.detach().cpu().numpy()

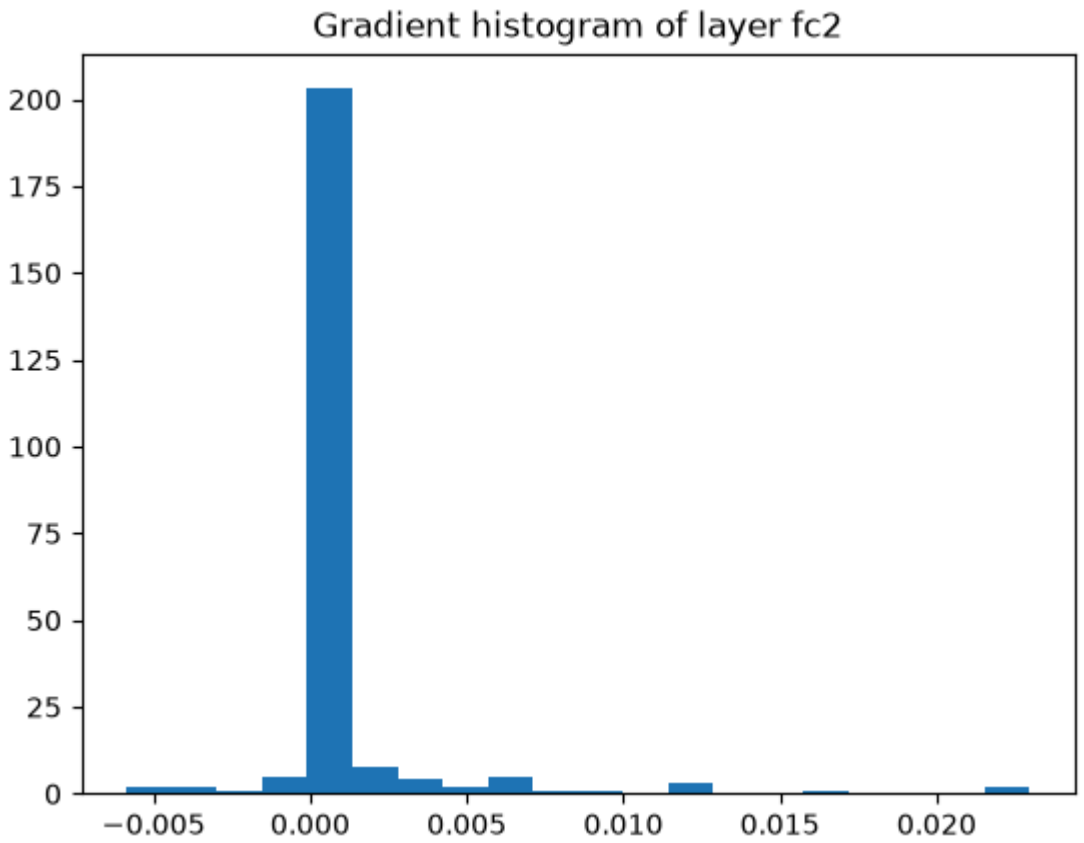
        # Reshape for histogram
        gradient = gradient.reshape(-1)
        _ = plt.hist(gradient, bins=20)
        plt.title("Gradient histogram of layer "+name)
        plt.show()
```


Gradient histogram of layer conv1
1200
1000
800
600
400
200
0
−0.4
−0.2
0.0
0.2
0.4


Gradient histogram of layer conv2
2000
1500
1000
500
0
−0.4
−0.2
0.0
0.2
0.4


Gradient histogram of layer conv3
17500
15000
12500
10000
7500
5000
2500
0
−0.4
−0.2
0.0
0.2
0.4


Gradient histogram of layer fc1
5000
4000
3000
2000
1000
0
−0.4
−0.2
0.0
0.2
0.4



In part **(b)**, the model uses random initial weights, so each filter/neuron produces different activations and receives different gradients. The gradient histograms therefore show a spread of values around zero.

In part **(c)**, all weights are set to zero. This makes many activations identical or zero, and during backpropagation the zero weights reduce or block gradient flow to earlier layers. As a result, the gradient histograms are much more concentrated near zero, and some layers may have all-zero gradients.

Initializing a CNN with zero weights is harmful because it causes symmetry: filters in the same layer start the same and receive the same updates, so they cannot learn different features. Training becomes very slow or may fail completely, especially in deeper layers.